Linear Regression from Scratch

OLS + SGD — Notes, Derivations & Code

by Nikhil Bisht | BCA Data Science Final Year

Handwritten Notes

Date
Page No.

1> Linear Regression

- Revision Notes / Short Notes :>
- it is a supervised machine learning Algo, which is of three types { Simple LR, multiple LR, Polynomial LR }

SLR → { independent col → 1, ~~but~~ dependent col → 1 }
MLR → { multiple input features → >1, 1 dependent col }

- Here we should have linear data { ~~not~~ in real life it will not be completly linear → almost linear }

⇒ Linear Regression is an algo which tries to fit a line in our data, or tries to find the relation between our dependent and independent variables by fitting a linear line.

it use ~~math~~ mathematical intution of line

SLR → $y = mx + c$

MLR ⇒ $y = m_1x_1 + m_2x_2 + m_3x_3 \dots m_nx_n + c$.

SLR

- How to find the value of (m, c)
  - we use gradient descent
  - OLS ⇒ Ordinary Least Square. { used in SKlearn }

- OLS formulaes ⇒ $b = \bar{y} - m\bar{x}$ → $\bar{y}$ → mean of target, $\bar{x}$ → mean of independent

$$m = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2}$$

$$\text{Error}(J) = \sum_{i=1}^{n} (y_i - \hat{y})^2$$

x_i = Row Value of x_i feature

y_i = Row Value of y_i

Date
Page No.

MLR → Here also everything remains same, ~~just~~ just the dimensions of input is increased.

for 2D → line, 3D → Plane, 4D - hyperplane

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 \cdots \beta_n x_n$$

Here we will find best value for hyperplane, in order to minimise the Loss

parameters

$$\hat{y} \Rightarrow \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \begin{bmatrix} \beta_0 + x_{11}\beta_1 + x_{12}\beta_2 \cdots x_{1m}\beta_m \\ \\ \beta_0 + x_{n1}\beta_1 + x_{n2}\beta_2 \cdots x_{nm}\beta_m \end{bmatrix}$$

$$\hat{y} = \underset{[n\times 1]}{\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}} = \underset{[n\times m]}{\begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1m} \\ 1 & x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix}} \underset{[m\times 1]}{\begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_m \end{bmatrix}}$$

$\hat{y} \Rightarrow A \qquad B$

$$\hat{y} = AB \text{ or } X\beta$$

$$\boxed{\hat{y} = XB}$$

$$y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} \quad \hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \vdots \\ \hat{y}_n \end{bmatrix} \quad e = \begin{bmatrix} y_1 - \hat{y}_1 \\ y_2 - \hat{y}_2 \\ \vdots \\ y_n - \hat{y}_n \end{bmatrix}$$

in SLR $(J)E = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$

in MLR $E(J) = e^T e$ [in terms of matrix]

$E = e^T e = (y-\hat{y})^T \cdot (y-\hat{y})$

$(A \pm B)^T = A^T \pm B^T$

$= (y^T - \hat{y}^T) \cdot (y - \hat{y})$; $\hat{y} = X\beta$

$= y^T \cdot y - y^T\hat{y} - \hat{y}^T \cdot y + \hat{y}^T \cdot \hat{y}$

$\hookrightarrow [\, y^T\hat{y} = \hat{y}^T \cdot y \,]$

$= y^T \cdot y - 2y^T\hat{y} + \hat{y}^T \cdot \hat{y}$

$\hookrightarrow$ imputing $\hat{y}$ value

$$E(J) = \boxed{y^T \cdot y - 2y^T \cdot (X\beta) + X^T\beta^T \cdot (X \cdot \beta)}$$

Proof of $y^T\hat{y} = \hat{y}^T \cdot y$

$\Downarrow$

assumming $y = A$, $\hat{y} = B$

$A^T B = B^T \cdot A$

$\hookrightarrow$ if we prove $B^T \cdot A = A^T B$, then $y^T\hat{y} = \hat{y}^T \cdot y$

$A^T B = (A^T B)^T = B^T A$ $\quad (A^T B) = Z$

$Z = (Z)^T$ $\quad Z = y^T\hat{y}$

$y^T X\beta = (y^T X\beta)^T$

$\underset{(m,1)}{y}^T \cdot \underset{(n,m)(m,1)}{X\beta}$

$(1 \times m) \quad (n \times 1)$

$\hookrightarrow$ output $= [1 \times 1] =$ output^T

$[-] = [-]^T$ proved

$$\frac{\partial J}{\partial \beta} = \frac{d}{d\beta}\left(\cancelto{0}{y^T y} - 2y^T X\beta + \beta^T X^T X\beta\right)$$

$$= -2y^T X + \frac{d(\beta^T y^T X\beta)}{d\beta}$$

$$= -2y^T X + 2X^T X\beta^T = 0$$

$\cdot\ \beta^T = \dfrac{\cancel{2}y^T X}{\cancel{2}X^T X}$

$\cdot\ \beta^T = (y^T X) \cdot (X^T X)^{-1}$

$$(\beta^T) = ((y^T x)(x^T x)^{-1})^T$$

$$\beta = ([x^T x]^{-1})^T \cdot (y^T x)^T$$

$$[A^T A]^T = A^T \cdot A$$

Same shape

$$\beta = ([x^T \cdot x]^{-1})^T \cdot x^T y$$

$$\beta = (x^T x)^{-1} \cdot x^T y$$

Let's implement MLR using SGD

Step 1 → initilize the parameters
step 2 → run a loop for epoch
Step 3 → use gradient descent to update the parameters

gradient calculation ⇒
for weight $\frac{dJ}{dw}$
for intercept/bias = $\frac{dJ}{db}$

$J = (y_i - \hat{y})^2$

$$\frac{dJ}{dw} = \frac{d(y_i - (wx + b))^2}{dw}$$
$$= 2(y_i - \hat{y}).(-x)$$
$$= -2(y_i - \hat{y}) \cdot x$$

$$\frac{dJ}{db} = 2(y_i - \hat{y})$$

Results — OLS vs SGD vs sklearn

```
[89]: y_pred_sklearn = sklearn_model.predict(X_test)

      print(f'sklearn ki model k coeff : {sklearn_model.coef_} \n and uska intercept : {sklearn_model.intercept_}  %')
      print(f'sklearn ki test MSE is {mean_squared_error(y_test,y_pred_sklearn)} ')
      print(f'sklearn ki test MAE is {mean_absolute_error(y_test,y_pred_sklearn)} ')
      print(f'sklearn ki test R2_score is {r2_score(y_test,y_pred_sklearn):.3f} %')

      print('\n','*'*110,'\n')

      y_pred_bhai_k_ols_model_se = model_bhai_ka_ols_wala.predict(X_test)

      print(f'Bhai ke ols wle model k coeff : {model_bhai_ka_ols_wala.coeff} \n and uska intercept : {model_bhai_ka_ols_wala.intercept}  %')
      print(f'Bhai ke ols wle model ki test MSE is {mean_squared_error(y_test, y_pred_bhai_k_ols_model_se):.4f}')
      print(f'Bhai ke ols wle model ki test MAE is {mean_absolute_error(y_test, y_pred_bhai_k_ols_model_se):.4f}')
      print(f'Bhai ke ols wle model ki test R2_score is {r2_score(y_test, y_pred_bhai_k_ols_model_se):.3f}')

      print('\n','*'*110,'\n')

      y_pred_bhai_k_SGD_model_se = model_bhai_ka_sgd_wala.predict(X_test)

      print(f'Bhai ke sgd wle model ki model k coeff : {model_bhai_ka_sgd_wala.coeff} \n and uska intercept : {model_bhai_ka_sgd_wala.intercept}  %')
      print(f'Bhai ke sgd wle model ki test MSE is {mean_squared_error(y_test, y_pred_bhai_k_SGD_model_se):.4f}')
      print(f'Bhai ke sgd wle model ki test MAE is {mean_absolute_error(y_test, y_pred_bhai_k_SGD_model_se):.4f}')
      print(f'Bhai ke sgd wle model ki test R2_score is {r2_score(y_test, y_pred_bhai_k_SGD_model_se):.5f}')

sklearn ki model k coeff : [98.3559076  82.80473759 25.83193455]
 and uska intercept : -0.044732055521219394  %
sklearn ki test MSE is 100.48489027843269
sklearn ki test MAE is 8.238693553190242
sklearn ki test R2_score is 0.994 %

 ****************************************************************************************************************

Bhai ke ols wle model k coeff : [98.3559076  82.80473759 25.83193455]
 and uska intercept : -0.04473205552122267  %
Bhai ke ols wle model ki test MSE is 100.4849
Bhai ke ols wle model ki test MAE is 8.2387
Bhai ke ols wle model ki test R2_score is 0.994

 ****************************************************************************************************************

Bhai ke sgd wle model ki model k coeff : [99.9197792  82.73584149 27.77214837]
 and uska intercept : -0.6195388989100108  %
Bhai ke sgd wle model ki test MSE is 104.1933
Bhai ke sgd wle model ki test MAE is 8.3759
Bhai ke sgd wle model ki test R2_score is 0.99355
```

Both OLS and SGD matched sklearn — $R^2 = 0.994$

Full Implementation — Python Code

```python
import numpy as np

class Bhai_ka_LinearRegression():
    """
    Linear Regression from scratch.
    Supports OLS (closed-form) and SGD (iterative) optimizers.

    Parameters
    ----------
    optimizer : str   — 'OLS' or 'SGD'
    epoch     : int   — number of training epochs (SGD only)
    lr        : float — learning rate (SGD only)
    """

    def __init__(self, optimizer='OLS', epoch=500, lr=0.01):
        self.optimizer = optimizer
        self.lr        = lr
        self.epoch     = epoch
        self.coeff     = None
        self.intercept = None

    def initilize_params(self, X):
        """Initialize weights to zero before SGD training."""
        n_features     = X.shape[1]
        self.coeff     = np.zeros(n_features)
        self.intercept = 0

    def ols(self, X, y):
        """
        OLS closed-form: beta = (X^T X)^-1 . X^T . y
        Prepends a bias column of 1s so intercept is part of beta.
        """
        X              = np.insert(X, 0, 1, axis=1)
        betas          = np.linalg.inv(np.dot(X.T, X)).dot(X.T).dot(y)
        self.intercept = betas[0]
        self.coeff     = betas[1:]

    def SGD(self, X, y):
        """
        Stochastic Gradient Descent — one random sample per update.

        Gradients:
            dJ/dw = -2*(yi - yhat)*xi
            dJ/db = -2*(yi - yhat)

        Update:
            w = w - lr * dJ/dw
            b = b - lr * dJ/db

        Prints MSE loss every 100 epochs.
        """
        self.initilize_params(X)

        for i in range(self.epoch):
            for j in range(X.shape[0]):
                idx  = np.random.randint(0, X.shape[0])
                Xi   = X[idx]
                yi   = y[idx]
                yhat = self.intercept + np.dot(Xi, self.coeff)

                Dcoeff     = -2 * (yi - yhat) * Xi
                Dintercept = -2 * (yi - yhat)

                self.coeff     = self.coeff     - self.lr * Dcoeff
```

```
                self.intercept = self.intercept - self.lr * Dintercept

            if i % 100 == 0:
                y_pred = self.intercept + np.dot(X, self.coeff)
                print(f'Epoch {i:>4} | MSE: {np.mean((y - y_pred)**2):.4f}')

    def fit(self, X, y):
        """Train using selected optimizer."""
        if self.optimizer == 'OLS':
            self.ols(X, y)
        if self.optimizer == 'SGD':
            self.SGD(X, y)

    def predict(self, X):
        """Return predictions for input X."""
        return self.intercept + np.dot(X, self.coeff)


# ■■ Quick Test ■■■
```

Links

Medium Article:

https://medium.com/@nikhilbisht226/i-built-linear-regression-from-scratch-in-python-ols-sgd-ec8e8c9a30a1

GitHub Repo:

https://github.com/nikhil1913/WOW_machine_learning